FSM Controllers

Contemporary Digital Electronic Systems — Chapter 15

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. Characteristics
3. Vending Machine
4. One-Hot Encoding
5. Zebra Crossing
6. Traffic Light
7. Summary

Why This Chapter Matters

A controller is an FSM that drives something else: a motor, a set of lights, a timer. Compared with Chapter 13's small examples, controllers have many inputs (sensors, requests, status flags) and many outputs (enables, loads, direction bits). The design procedure is the same; there is just more to track.

After this chapter, you can:

1. Translate a multi-input, multi-output specification into a state diagram.
2. Design a controller for a traffic light, a vending machine or a pedestrian crossing.
3. Pair a controller with a register-based circuit such as a timer, linked by control and status signals.
4. Use a timer built from a counter or prescaler to drive controller inputs on a slower timebase.

Scope. The circuits controlled are basic register-based blocks (flip-flops, counters, timers) and simple actuators. They store or count but do not process data, so the controller only sequences them.

© Muním Zabidi

An Opening Question

A vending machine takes coins and must dispense when the running total reaches the price. What states does it need, and what inputs trigger each transition?

The chapter works the nickel/dime machine (15 cents) in detail.

Roadmap

1. Vending machine: multiple inputs and outputs; Moore, then Mealy, then one-hot encoding.
2. Zebra crossing: adds a timer subsystem that supplies a slower timebase.
3. Traffic light: reuses the timer idea in a larger controller with four states.
4. An elevator controller is left as an end-of-chapter exercise.

Table of Contents

1 Why This Chapter

2 **Characteristics**

3 Vending Machine

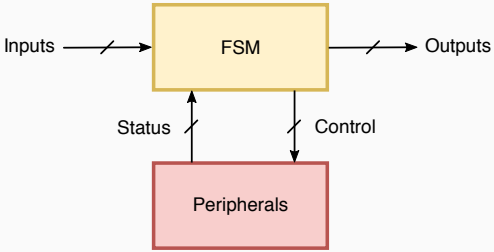
4 One-Hot Encoding

5 Zebra Crossing

6 Traffic Light

7 Summary

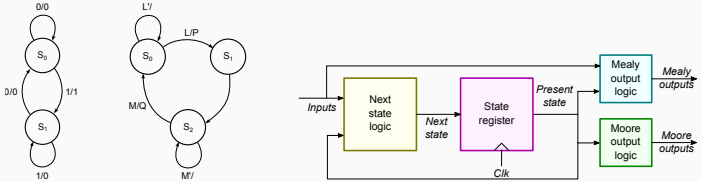
Characteristics of Controllers



- An FSM that interfaces with peripherals or supporting circuits.
- Outputs control external devices; status signals come back from peripherals; other inputs come from the environment.
- May have both Moore and Mealy outputs.

© Munim Zabidi

Many Inputs, Many Outputs



- In multi-input diagrams, arrows are labelled with Boolean conditions on named signals instead of 0 and 1.
- Use Moore outputs where possible (easy to analyse). Use Mealy outputs where a pulse is needed without a dedicated state. Mixing both can simplify the design.

Table of Contents

1 Why This Chapter

2 Characteristics

3 **Vending Machine**

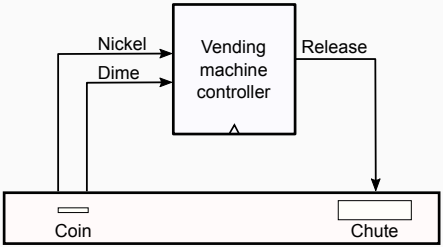
4 One-Hot Encoding

5 Zebra Crossing

6 Traffic Light

7 Summary

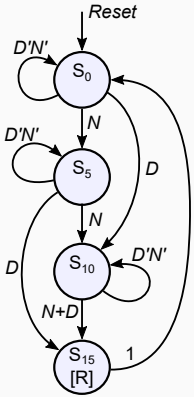
Vending Machine Controller



- One coin slot; nickels (5 cents) and dimes (10 cents), one at a time. A sensor identifies the coin.
- Releases the product at 15 cents. No change: two dimes lose 5 cents.
- Inputs *N*, *D* (both 1 never occurs); output *Release*.



Moore Vending Machine: State Diagram



- S_0: no money; S_5, S_{10}, S_{15}: that amount received.
- A nickel moves forward one state; a dime moves two. No coin: stay (loop $D'N'$).
- In S_{15} the machine asserts *Release*, then returns to S_0 unconditionally (arrow labelled 1).
- Arrows carry Boolean conditions such as $D'N'$, $N + D$, D.

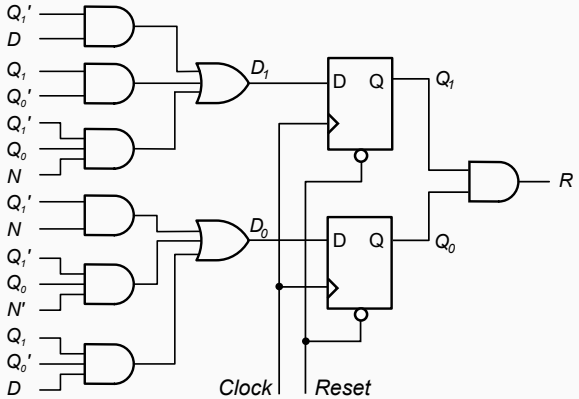
Moore Vending Machine: State Table

State	D	N	Next	*Release*
S_0	0	0	S_0	0
	0	1	S_5	0
	1	0	S_{10}	0
	1	1	×	0
S_5	0	0	S_5	0
	0	1	S_{10}	0
	1	0	S_{15}	0
	1	1	×	0
S_{10}	0	0	S_{10}	0
	0	1	S_{15}	0
	1	0	S_{15}	0
	1	1	×	0
S_{15}	×	×	S_0	1

- $DN = 11$ is impossible: don't cares.
- Binary encoding: $S_0 = 00$, $S_5 = 01$, $S_{10} = 10$, $S_{15} = 11$.

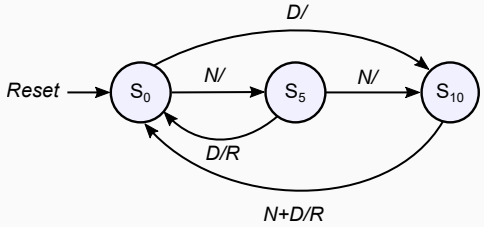
$$D_1 = Q_1'D + Q_1Q_0' + Q_1'Q_0N$$
$$D_0 = Q_0'N + Q_1'Q_0N' + Q_1Q_0'D$$
$$R = Q_1Q_0$$

Moore Vending Machine: Circuit



- Two flip-flops with next-state logic built from the equations; *R* comes straight from the state bits.

Mealy Vending Machine



- No S_{15} state: R is produced during the transition from S_5 or S_{10} back to S_0.
- Three states, encoding $S_0 = 00$, $S_5 = 01$, $S_{10} = 10$.

$$D_1 = Q_0 N + Q_1' Q_0' D + Q_1 D' N'$$
$$D_0 = Q_1' Q_0' N + Q_0 D' N'$$
$$R = Q_0 D + Q_1 N + Q_1 D$$

- Literals: 20 against the Moore version's 17, mostly in R (six literals, since it depends on the inputs as well as the state). One fewer state, but here that does not pay for the extra output logic, so the implementation is skipped.

Table of Contents

1 Why This Chapter

2 Characteristics

3 Vending Machine

4 **One-Hot Encoding**

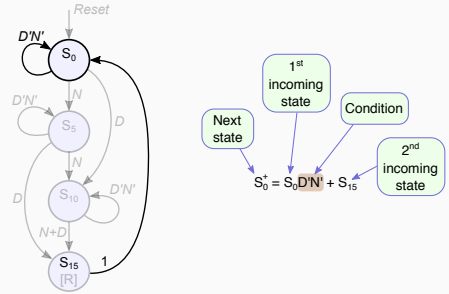
5 Zebra Crossing

6 Traffic Light

7 Summary

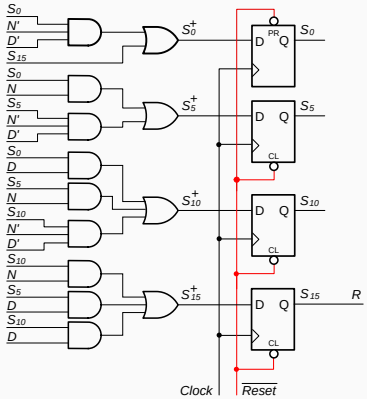
One-Hot Moore Vending Machine

- One flip-flop per state; only the current state's flip-flop is "hot".
- $S_0 = 0001, S_5 = 0010, S_{10} = 0100, S_{15} = 1000$.
- Each incoming arrow gives one term: originating state AND the arrow's condition. No Boolean reduction needed.



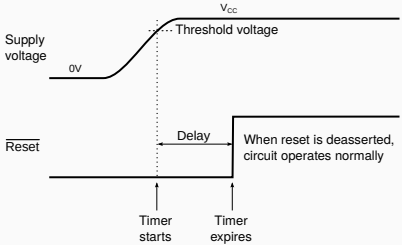
$$S_0^+ = S_0 D' N' + S_{15}$$
$$S_5^+ = S_0 N + S_5 D' N'$$
$$S_{10}^+ = S_0 D + S_5 N + S_{10} D' N'$$
$$S_{15}^+ = S_5 D + S_{10} N + S_{10} D$$
$$R = S_{15}$$

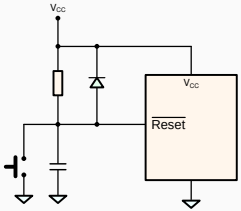
One-Hot Circuit and Reset



- $\overline{Reset}$ drives the preset of the first flip-flop and the clear of the others, forcing S_0.
- One-hot logic is not limited to four input variables and is easy to extend.

Automatic Reset: Power-On Reset





- A real system must start without human help. A power-on-reset circuit holds reset low while the supply rises. A timer starts when the supply crosses a threshold; POR is released high only when the timer expires.
- Until then all flip-flops are forced to the initial state.

© Muním Zabidi

Automatic Reset: Inverted First Flip-Flop

- In CPLDs and FPGAs all flip-flops power up as 0. For a highly encoded FSM, all-zero is a fine initial state; for one-hot it is the “none-hot” state.
- Fix: use negative logic on the first flip-flop (an inverter at its input and output). Then all-zeros means S_0.

State	Actual FFs	Reads as
S_0	0000	0001
S_5	0011	0010
S_{10}	0101	0100
S_{15}	1001	1000

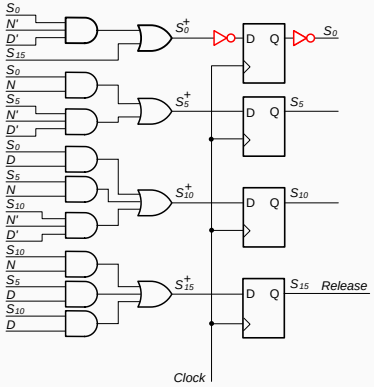


Table of Contents

1 Why This Chapter

2 Characteristics

3 Vending Machine

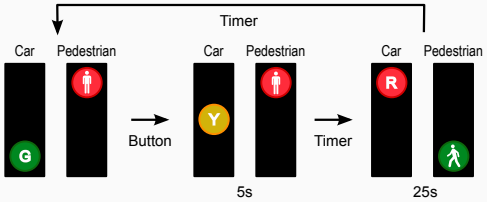
4 One-Hot Encoding

5 **Zebra Crossing**

6 Traffic Light

7 Summary

Zebra Crossing Controller

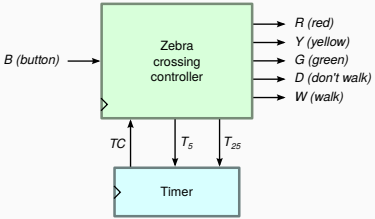


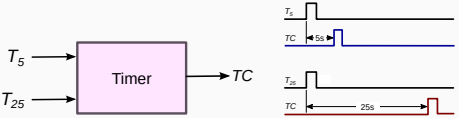
- Adds a timer subsystem: the controller starts the timer with a control signal and waits for its status signal.
- Car lights *R*, *Y*, *G*; pedestrian lights *W* (walk), *D* (don't walk); input *B* (button).
- Cars have right of way by default. A press gives 5 s of yellow, then 25 s of red/walk.

State	Car	Pedestrian	Period
Initial	Green	Don't walk	Infinite
Transition	Yellow	Don't walk	5 s
Safe to walk	Red	Walk	25 s

© Muním Zabidi

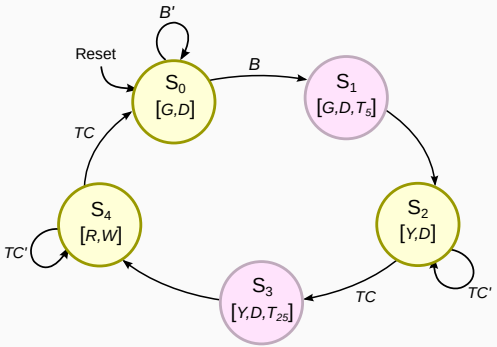
Zebra Crossing: Interfaces and Timer



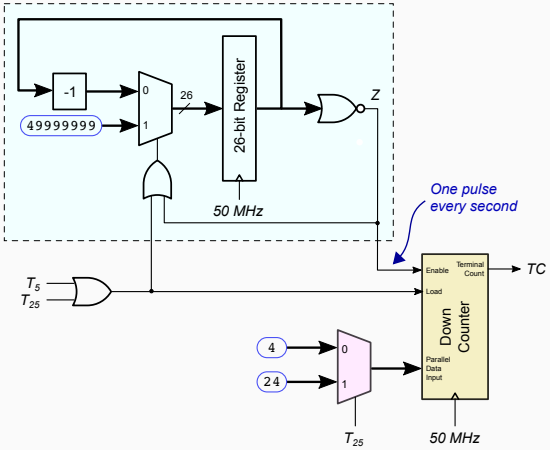


- The controller sends T_5 or T_{25} to the timer. After 5 s or 25 s the timer asserts *TC* (terminal count).
- The timer is internal; its signals are not visible outside.

First-Cut Moore Design



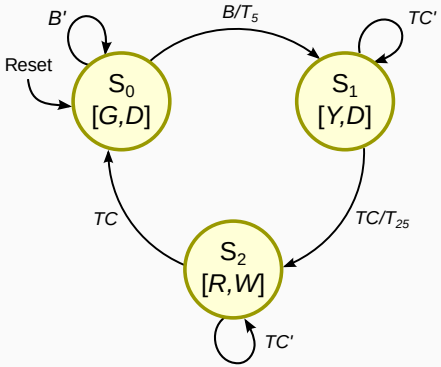
- S_0: G and D on; wait for $B = 1$.
- S_1: one cycle that asserts T_5 to trigger the timer.
- S_2: yellow for 5 s; wait for TC.
- S_3: one cycle that asserts T_{25}.
- S_4: red and walk for 25 s; on TC, back to S_0.
- Two states exist only to trigger the timer: in a pure Moore machine an output must live inside a state.



- A 1 Hz timing reference (the prescaler of Section 11.4.4) pulses Z once per second. At 50 MHz it counts down from 49,999,999.
- A 5-bit down-counter with load and enable counts seconds; loaded with 4 or 24 when T_5 or T_{25} arrives.
- Loading n gives an $(n + 1)$-second delay, because TC is seen on the clock after the count reaches 0.

© Muním Zabidi

Improved Design: Mixed Moore and Mealy



State	B	TC	Next	D	W	R	Y	G	T_5	T_{25}
S_0	0	×	S_0	1	0	0	0	1	0	0
S_0	1	×	S_1	1	0	0	0	1	1	0
S_1	×	0	S_1	1	0	0	1	0	0	0
S_1	×	1	S_2	1	0	0	1	0	0	1
S_2	×	0	S_2	0	1	1	0	0	0	0
S_2	×	1	S_0	0	1	1	0	0	0	0

- Triggering the timer on a transition (Mealy output) removes the two trigger states: three states instead of five.

© Muním Zabidi

Zebra Crossing: Equations and Circuit

One-hot next state:

$S_0^+ = S_0B' + S_2TC$

$S_1^+ = S_0B + S_1TC'$

$S_2^+ = S_1TC + S_2TC'$

Moore outputs: $D = S_0 + S_1$, $W = R = S_2$, $Y = S_1$, $G = S_0$.

Mealy outputs: $T_5 = S_0B$, $T_{25} = S_1TC$.

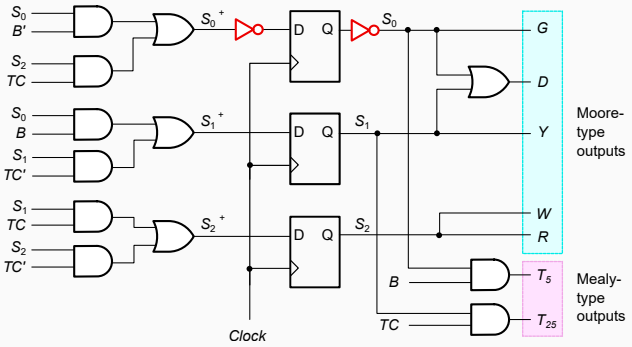


Table of Contents

1 Why This Chapter

2 Characteristics

3 Vending Machine

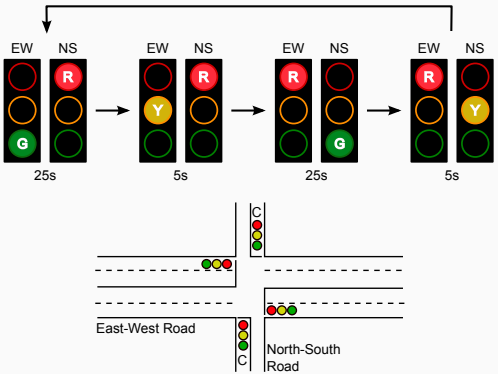
4 One-Hot Encoding

5 Zebra Crossing

6 **Traffic Light**

7 Summary

Traffic Light Controller

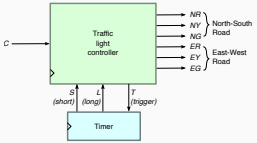


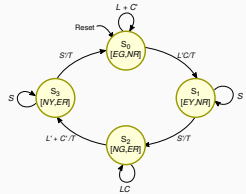
- Reuses the timer idea of the zebra crossing, with more states and timed phases.
- East-West has priority (green by default); North-South has a car sensor C.
- A car on North-South triggers: EW yellow 5 s, then NS green (at most 25 s), then NS yellow 5 s, then back to EW green.

State	EW	NS	Delay
S_0	Green	Red	$\geq$ 25 s while $C = 0$
S_1	Yellow	Red	5 s
S_2	Red	Green	$\leq$ 25 s
S_3	Red	Yellow	5 s

© Muním Zabidi

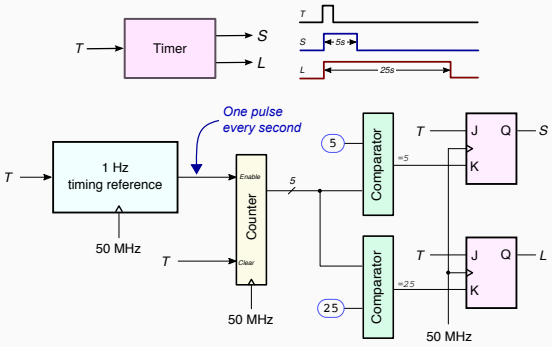
Traffic Light: Architecture and State Diagram





- Inputs: C (car sensor), S (short timer), L (long timer). Outputs: six lights and T (trigger).
- S_0: exit on $L'C$ (long timer expired and car present): at least 25 s.
- S_1: stay while $S = 1$; when $S = 0$ trigger the timer and go to S_2.
- S_2: stay only while L and C are both 1; otherwise go to S_3 and trigger the timer.
- S_3: stay while $S = 1$, then return to S_0.

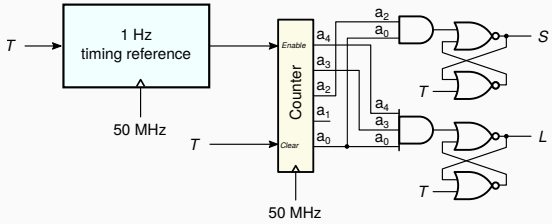
Traffic Light Timer Subsystem



- Trigger *T* sets both outputs *S* and *L* high; they go low after 5 s and 25 s.
- Library version: a counter, two comparators and two JK flip-flops. *T* resets the counter and the 1 Hz reference; the comparators detect 5 and 25 and reset the flip-flops.

© Munim Zabidi

Gate-Level Timer



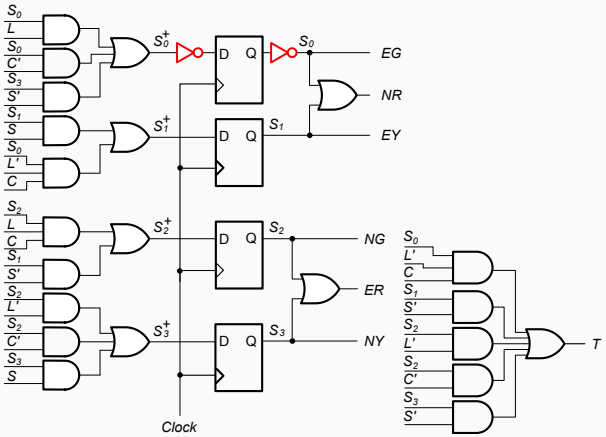
- Smaller version: S and L come from SR latches set by T.
- Because 5 and 25 are fixed, a gate detects them cheaply: a_2a_0 detects $xx1x1_2$, and $5 = 00101_2$ is the first such value reached.
- $a_4a_3a_0$ detects $11xx1_2$, and $25 = 11001_2$ is the first match.
- Each AND output resets its latch.

Traffic Light: One-Hot Implementation

One-hot states $S_0 = 0001$ to $S_3 = 1000$.

$$S_0^+ = S_0(L + C') + S_3S'$$
$$S_1^+ = S_0L'C + S_1S$$
$$S_2^+ = S_1S' + S_2LC$$
$$S_3^+ = S_2(L' + C') + S_3S$$

$T = S_0L'C + S_1S' + S_2(L' + C') + S_3S'$
$EG = S_0, EY = S_1, ER = S_2 + S_3$
$NG = S_2, NY = S_3, NR = S_0 + S_1$



Vending machine states and inputs

States: the amount received so far (S_0, S_5, S_{10}, and a release state S_{15} in the Moore design).
Inputs: N (nickel) and D (dime); $ND = 11$ is impossible.
Transitions: a nickel moves one step, a dime two; reaching 15 cents asserts *Release* and returns to S_0. For a different price or coin set, add one state per reachable total.

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Characteristics

3 Vending Machine

4 One-Hot Encoding

5 Zebra Crossing

6 Traffic Light

7 **Summary**

Summary

- A controller is an FSM that sequences operations on something else: a timer, a peripheral, the lights.
- Here the controlled circuits are register-based blocks (counters, timers) and simple actuators: no data processing.
- Many controllers mix Moore outputs (stable, used as enables and selects) with Mealy outputs (reactive, used for pulses).
- Prescalers and divided clocks give a controller a slower timebase without complicating the state diagram.
- Always define a reset state; every reachable state needs a transition for every input combination.

Looking ahead

Controllers can also work with *datapaths*, the circuits that process data, and this scales up to complete processors. Chapter 16 introduces Verilog for describing all of this.